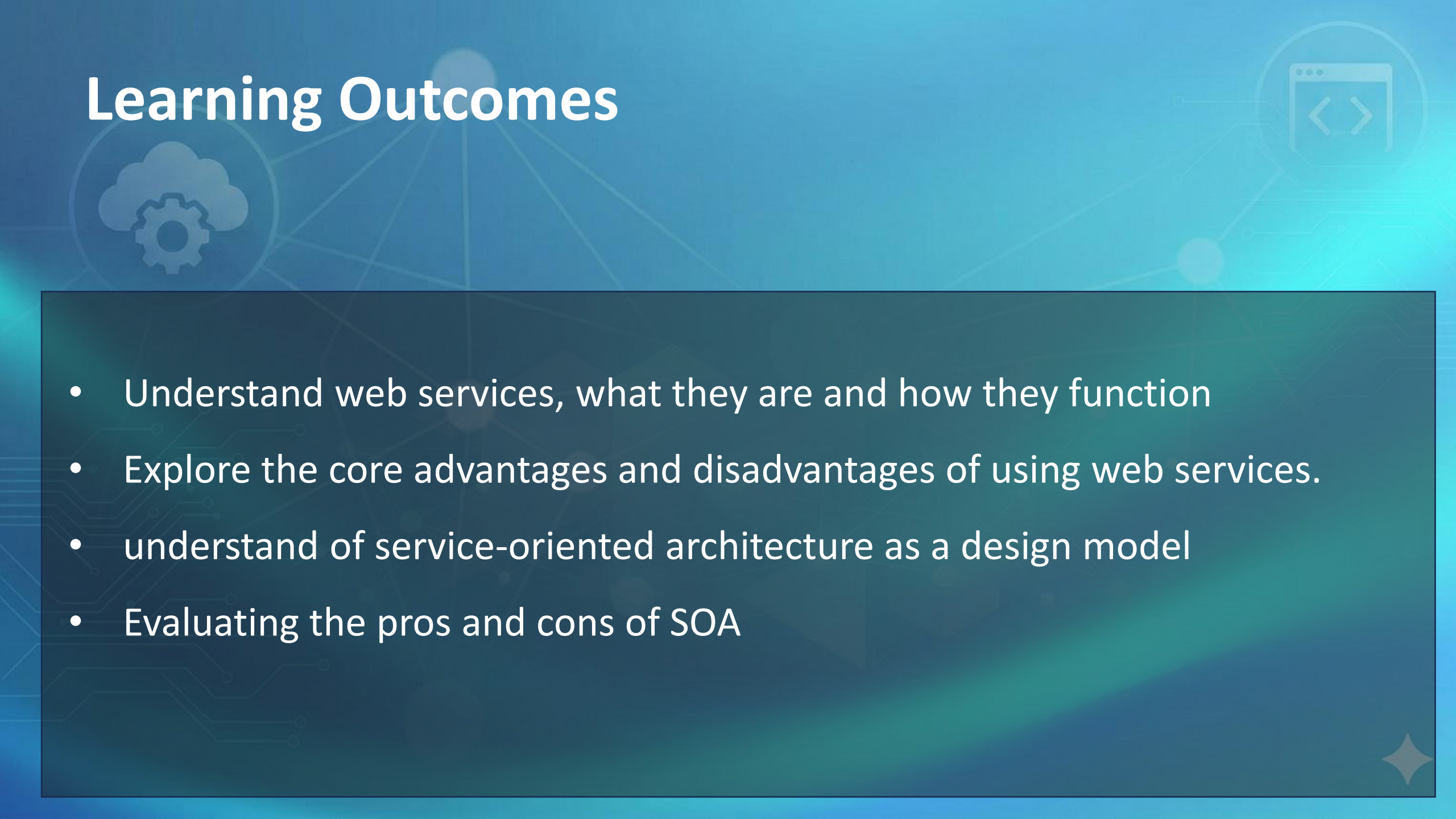


INTRODUCTION TO SERVICE ORIENTED COMPUTING (SOC)

From Web Services to Modern RESTful Architectures



Learning Outcomes



- Understand web services, what they are and how they function
 - Explore the core advantages and disadvantages of using web services.
 - understand of service-oriented architecture as a design model
 - Evaluating the pros and cons of SOA
- 

What is SOC?

- SOC is a distributed computing paradigm.
- It organizes systems into loosely coupled services.
- Each service is designed to perform a specific function.
- Services can be combined to build complex applications.

Business Tasks as Services

- Business processes can be modeled as services.
- Example: processing a funds withdrawal transaction.
- Example: handling an online purchasing workflow.
- Example: retrieving or updating database records.
- This approach improves reusability and consistency

The "Why" of SOC

- SOC enhances system efficiency through reuse of services.
- It enables seamless integration across different platforms.
- It supports automation of complex business processes.
- It reduces development time and operational overhead.

Two Flavors of Web Services

- Informational, or Type I, Web services support simple request and response operations.
- These services are discrete and always wait for a request before processing and responding.
- Complex, or Type II, Web services implement a form of coordination between inbound and outbound operations.
- This coordination allows services to handle more sophisticated, multi-step business interactions.

Classic SOC (SOAP/XML)

- The first generation of Web services architecture was built on the SOAP specification.
- SOAP is an XML-based messaging protocol used to structure communication between clients and services.
- It is particularly useful for performing RPC-style (Remote Procedure Call) request-response dialogues.
- Messages formatted in SOAP can contain method calls, required parameters, and security information in header items.

Transitioning to REST

- The industry moved toward REST to leverage the simplicity of existing Internet standard technologies.
- Traditional protocols like SOAP use plain text and a verbose method to identify data, which increases message size.
- REST improves performance by reducing the "extra size" issue often found in busy or low-speed connections.
- By using standard HTTP, REST simplifies the stack and removes the need for highly customized integration applications.

Core Principles of REST

- Resources are the primary building blocks and are identified by a unique URL, just like a website.
- Statelessness ensures that the server and client have no knowledge of each other between exchanges, making the system easier to scale.
- A Uniform Interface allows different platforms (like Java and Python) to interact dynamically using open standards.
- This architectural style promotes loose coupling, reducing the degree of dependency systems have on each other.

REST and HTTP Methods

- HTTP serves as the transport protocol to move requests and responses between clients and servers.
- Standard HTTP verbs are used to carry any data type from point A to point B regardless of the underlying implementation.
- The GET method is typically used to retrieve a document or information from a service.
- POST, PUT, and DELETE methods allow for the creation, updating, and removal of service-enabled resources

Data Representation: JSON vs. XML

Feature	XML	JSON
Structure	Uses custom tags (<tag>data</tag>).	Uses simple key-value pairs ("key": "value").
Size	Verbose; uses more characters, making requests larger.	Lightweight; uses fewer characters for the same data.
Performance	Can be an obstruction over low-speed or busy connections.	Faster to transmit and parse, making it ideal for modern services.
Focus	Originally designed for large-scale electronic publishing.	Designed for simple, high-performance data exchange.
Usage	The foundation for classic SOAP web services.	The industry standard for modern RESTful architectures.

REST and HTTP Methods

- The Service Provider publishes connection details so that the service can be utilized by consumers.
- In a RESTful world, API Documentation replaces the UDDI Service Registry to help developers find existing services.
- The Service Requester uses these details to perform a "Find" operation to locate the necessary functionality.
- Once the details are found, the requester performs a "Bind" operation to send messages and receive responses.

Interoperability

- Interoperability allows systems built with different technologies to communicate.
- Applications can exchange data across platforms and environments.
- example, Java-based and Python-based systems can interact.
- This is achieved through standardized communication protocols.

Loose Coupling

- The term coupling indicates the specific degree of dependency that any two systems have on each other.
- Web services use Internet standard technologies to build bridges between systems that would otherwise require extensive development.
- Loose coupling makes it possible for these systems to interact with one another dynamically.
- Reducing this dependency allows for service-oriented solutions to be edited or updated more easily.

Stateless Properties

- Stateless services can be invoked repeatedly without the need to maintain a specific context or state.
- If a service requires its context to be preserved from one invocation to the next, it is considered stateful.
- Modern service access protocols are usually connectionless, meaning they have no inherent concept of a job or a session.
- This property simplifies service design by not making assumptions about eventual delivery or session persistence.

Service Granularity

- Fine-grained descriptions regard the smaller components of which larger systems are composed.
- These simple services are discrete, atomic in nature, and usually exhibit a request/reply mode of operation.
- Coarse-grained descriptions regard large subcomponents or complex systems.
- Complex services are coarse-grained because they often access and combine information from multiple different sources.

Cost Savings & Re-usability

- One service might be utilized by several different clients to fulfill various business objectives.
- Re-usability allows portions of a service to be used as necessary instead of creating custom services for every unique requirement.
- Easy interoperability removes the expensive need to create highly customized applications for integrating data.
- Existing investments in infrastructure can be combined easily to add value and reduce the associated learning curve.

Security & Privacy

- Anything sent over the Internet can be viewed by others, making protection essential.
- The primary approved option for sending sensitive information to Web services is the Secure Socket Layer (SSL).SSL over HTTP, commonly known as HTTPS, is effective at safeguarding information such as credit card numbers.
- While easy to set up, this technology is slower than unencrypted transactions and may be unsuited for very large data transfers.

Extra Overload

- In a service-oriented architecture, all inputs must be validated before they are sent to a service.
- Using multiple services can overload a system with the extra computation required for these validations.
- Web services that handle frequent messaging can easily reach millions of requests per day.
- Running these services successfully requires high-speed servers with significant data bandwidth.